

PROD 배포 런북 — Get FYUSD / Earn / Redeem (Ethereum mainnet, chainId 1)

2026-06-17. 이 범위에는 epoch 및 FYUSD 토큰 컨트랙트가 포함되지 않습니다 (FYUSD는 BitGo 엔터프라이즈 발행으로 오프체인에서 처리되며, Get-FYUSD와 Redeem은 순수 BitGo 직접 커스터디 흐름입니다). 온체인 컨트랙트가 존재하는 것은 Earn (70:30)뿐입니다. 담보는 USDC와 USDT 둘 다 사용합니다 → earn vault 2개 + adapter 2개가 필요합니다.

외부 감사(audit)가 FyusdEarnVault / ConcreteStableAdapter / EarnLockRegistry 에 대해 통과되기 전까지는 mainnet에서 어떠한 실제 자금 흐름도 발생시키지 않습니다. 컨트랙트를 배포하는 것은 가능하지만, 감사 통과 전에는 실제 자금을 라우팅하지 마십시오.

배포할 컨트랙트 (의존성 순서)

1. SettingManagement (proxy) — initialize(deployer) ; 이후 admin 권한을 Operator Safe로 이전합니다.
2. ReservePool — constructor(SettingManagement) ; + SettingManagement.setReservePool(pool) .
3. FypherCircuitBreaker (proxy) — initialize(SettingManagement, watchdog) .
4. FyusdEarnVault × 2 (proxy, ERC-4626 = vFYUSD) — initialize(SM, stable, adapter, admin) . USDC용 하나, USDT용 하나. 감사 핵심 대상(audit-critical).
5. ConcreteStableAdapter × 2 — constructor(stable, concreteMainnetVault, vaultProxy) ; concreteVault.asset() 는 반드시 stable과 같아야 합니다.
6. EarnLockRegistry — constructor() + setLocker(relayer, true) .

배포하지 않는 것: FYUSD.sol, FyusdEpochSettlement/Redemption, FypherMinting/BurnQueue/StakingHub/FyusdYieldVault, RUSD/FYP/Staked*, lending, Mock*.

스크립트

- deploy-mainnet-core.js (신규) — SettingManagement + ReservePool + CircuitBreaker.
- deploy-fyusd-earn-vault.js — vault + adapter (+초기화). 두 번 실행 (USDC, USDT). 이름당 키를 하나만 기록하므로 → 두 쌍 모두 별도(out-of-band)로 캡처하십시오.
- deploy-earn-lock-registry.js — lock registry.
- deploy-operator-safe.js — Safe (또는 app.safe.global UI 사용). SAFE_THRESHOLD env 사용; owner 목록은 3개입니다 (3-of-5로 하려면 5개로 확장).
- grant-admin-to-safe.js / accept-admin-from-safe.js — 이제 체인 인식(chain-aware) 처리됩니다. mainnet에서의 acceptAdmin() → Safe UI에서 실행하십시오 (하드웨어 지갑 owner들).
- deploy-mainnet.js (깨짐 — scripts/deploy.js 누락) · deploy-phase1.js (범위에서 제외된 epoch/FYUSD/yield-vault를 배포함). 사용하지 마십시오.

순서대로 진행하는 체크리스트 (오후 1시)

```

cd sotatek-smart-contracts
# .env: PRIVATE_KEY=<자금이 들어 있는 mainnet deployer>, ETHERSCAN_API_KEY, MAINNET_RPC_URL=<private RPC>
npx hardhat compile

# 1) Operator Safe — app.safe.global (권장) 또는:
SAFE_THRESHOLD=3 npx hardhat run scripts/deploy-operator-safe.js --network mainnet
# → Safe 주소를 addresses/1.json (OperatorSafe) + .env (OPERATOR_SAFE_ADDRESS) + dashboard addresses.ts(1) 에 기록

# 2) Core (SM + ReservePool + CircuitBreaker)
WATCHDOG_ADDRESS=0x<guardian> npx hardhat run scripts/deploy-mainnet-core.js --network mainnet

# 3) Earn vault + adapter — USDC
STABLE_ADDRESS=0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48 \
CONCRETE_VAULT_ADDRESS=0x<Concrete mainnet USDC vault> \
ADMIN_ADDRESS=0x<deployer = 현재 SM admin> \
npx hardhat run scripts/deploy-fyusd-earn-vault.js --network mainnet
# → FyusdEarnVault(USDC) + ConcreteStableAdapter(USDC) 기록

# 3b) Earn vault + adapter — USDT
STABLE_ADDRESS=0xdAC17F958D2ee523a2206206994597C13D831ec7 \
CONCRETE_VAULT_ADDRESS=0x<Concrete mainnet USDT vault> \
ADMIN_ADDRESS=0x<deployer> \
npx hardhat run scripts/deploy-fyusd-earn-vault.js --network mainnet
# → FyusdEarnVault(USDT) + ConcreteStableAdapter(USDT) 기록

# 4) Lock registry
RELAYER_ADDRESS=0x<backend gas-relayer> npx hardhat run scripts/deploy-earn-lock-registry.js --network mainnet

# 5) Admin → Safe (2단계)
npx hardhat run scripts/grant-admin-to-safe.js --network mainnet # transferAdmin(safe) — 되돌릴 수 있음
# 그다음 app.safe.global (eth:<safe>)에서: SettingManagement.acceptAdmin() — threshold 만큼의 owner로 서명
# → Safe가 모든(ALL) 컨트랙트의 admin이 됩니다 (각 컨트랙트는 SM.hasRole을 실시간으로 조회해 admin을 확인)
# 추가로: EarnLockRegistry.setOwner(safe) (Safe tx)

# 6) 배포 후 작업 (Safe에서 실행): vault별 — setKeeper(<backend hot wallet>), setPauserRole(<guardian|circuitBreaker>),
# SettingManagement.setPoolConfigs("vFyusdEarnCooldown", 1209600). Concrete가 우리 adapter 주소 두 개를 모두 화이트리스트에 등록

# 7) 검증: npx hardhat verify --network mainnet <addr> <ctor args> (adapters, ReservePool, EarnLockRegistry)

```

백엔드 (gateway) 설정

```

FYPERX_CHAIN_ID=1 · ETH_RPC=<mainnet RPC>
FYPERX_ADMIN_TX_MODE=safe-propose
FYPERX_OPERATOR_SAFE_ADDRESS=<safe>
FYPERX_SAFE_TX_SERVICE_URL=https://api.safe.global/tx-service/eth · FYPERX_SAFE_CHAIN_SLUG=eth
FYPERX_EARN_VAULT_ADDRESS=<vault(s)> · FYPERX_EARN_COLLATERAL_SYMBOL=USDC|USDT
BETA_ALLOCATION_FYUSD_BPS=7000 (70:30, 백엔드 관리, 온체인 아님)
BACKEND_GAS_RELAYER_PRIVATE_KEY=<relayer; api.safe.global/tx-service/eth 에 Safe delegate로 등록>

```

팀이 담당하는 블로커 (배포 전에 해결)

1. Concrete mainnet vault 주소 × 2 (USDC + USDT) + 배포 후 Concrete가 우리 adapter 주소 두 개를 화이트리스트에 등록.
2. Safe owner 목록 + threshold (ADR-007 → 3-of-5; deploy-operator-safe의 owner 목록은 3개 → 5개로 확장하거나 Safe UI 사용).
3. 자금이 들어 있는 mainnet deployer 키 (PRIVATE_KEY) — 약 8회 배포 + admin tx에 필요한 ETH.
4. 외부 감사 게이트(External audit gate) 가 FyusdEarnVault / ConcreteStableAdapter / EarnLockRegistry에 대해 통과 완료.

5. MAINNET_RPC_URL — private provider 설정 (llamarpc 기본값은 주소 해석(resolution)에만 적합).